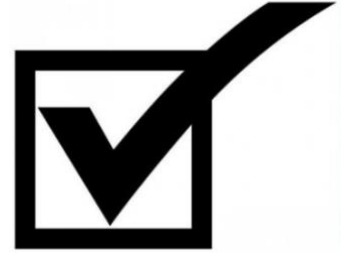




**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential



C++ PROGRAMMING

(335)

NATIONAL 2022

APPLICATION KNOWLEDGE:

Depth-First Maze Pathfinder _____ (500 points)

TOTAL POINTS _____ (500 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

Depth-First Maze Pathfinder

In this exercise, you will create a C++ console application that performs the depth-first search algorithm on a 2D ASCII map, and expanding that algorithm to make it a simple pathfinder. Pathfinding algorithms are commonly used in video games to provide NPC AI a means to move around the game world.

The application will first load the level from “map.txt”, render with the player’s start position, perform the pathfinding algorithm, display where the end was found, and then display the path to reach the end from the start position.

```
@..#o
#..#.
..##.
.#...
...##

End found at 4, 0

Step 1: 0, 0
Step 2: 1, 0
Step 3: 1, 1
Step 4: 1, 2
Step 5: 0, 2
Step 6: 0, 3
Step 7: 0, 4
Step 8: 1, 4
Step 9: 2, 4
Step 10: 2, 3
Step 11: 3, 3
Step 12: 4, 3
Step 13: 4, 2
Step 14: 4, 1
Step 15: 4, 0
```

Figure 1: Output from application, showing first the ASCII map, then where the end of the map is, and then the steps to get there.

Note that a ‘.’ represents a floor tile (traversable), ‘@’ represents player start position (traversable), ‘#’ represents a wall tile (not traversable), and ‘o’ represents where the end of the map is.

Depth-first searching works by, in a graph or some other structure, starting at a set location and checking one of the immediately adjacent traversable locations first, and then from that location digging deeper into its immediately adjacent traversable locations. This is done until traversal is unable to go any further in that direction – at that point the starting location’s next adjacent traversable location is checked and similarly traversed until the desired node is found.

In order to perform pathfinding off a searching algorithm like this, a list of traversed nodes that resulted in the desired node will need to be created throughout the traversal algorithm.

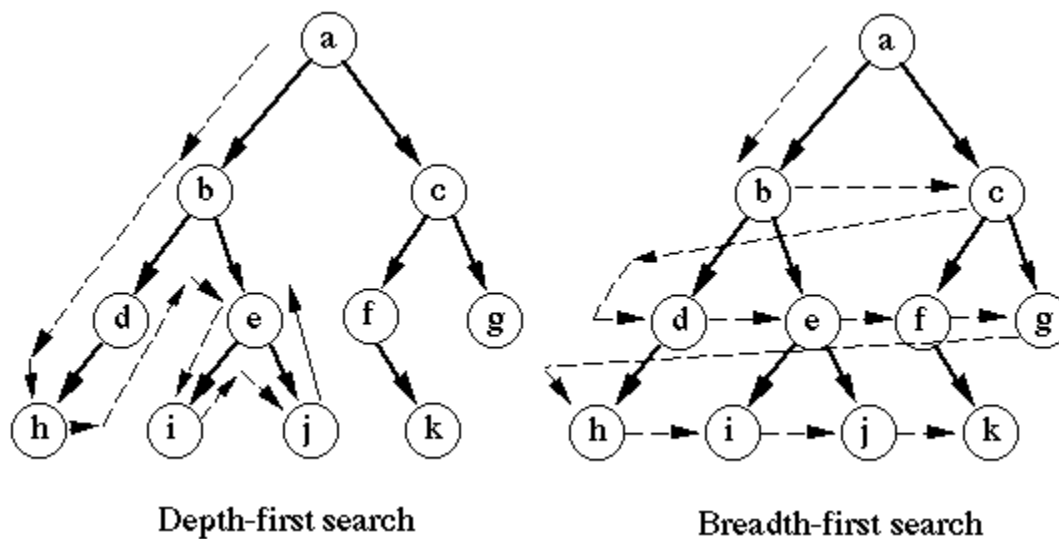


Figure 2: Traversal differences between DFS and BFS

Requirements:

1. You must create a C++ console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. On start, application shall load the map and locate the start position
 - a. If the file was not able to be opened, application shall show an error message indicating that there was an error
 - b. If the data structure of choice is empty after the file is loaded, application shall display an error message that there were no data elements found.
4. Once map has been loaded successfully it shall be rendered and the search and pathfinding algorithm shall be invoked.
5. When application displays where the end is, it must be an output from the depth-first search and pathfinding algorithm. Solutions that use 2D array traversal to find the end will not be accepted.
6. A level class shall be created that houses an appropriate data type for housing a 2D ASCII map with variable sizing.
 - a. Application may be tested with files other than included “map.txt”
 - b. A member function render shall be provided to display the ASCII map
 - c. A member load function shall be provided to load a passed file and output player start position
 - i. This function shall return false in the event of errors so application can handle error conditions, such as inability to open
 - d. A static traverse function shall be provided to perform the depth-first search and pathfinding algorithm
 - i. This function must be done recursively – any iterative solutions will not be accepted.
 - ii. Depth-first search must be used. Breadth-first, A*, or Dijkstra’s algorithm will not be accepted.
 - iii. This function shall construct only a single path – traversal shall cease after first path reaches end of map.
7. A helper struct coords shall be created to store positional data in the path data structure

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____ 10 pts
The project is named according to the naming conventions	_____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB	_____ 10 pts
--	--------------

If the program does not execute, then the remaining items in this section receive a score of zero.

Application displays error message if cannot load file	_____ 20 pts
Application displays error message if no elements in loaded file	_____ 20 pts
Application successfully loads map from file and displays it	_____ 30 pts
Application properly displays searched end of map	_____ 40 pts
Application properly displays single, correct path to end of map	_____ 50 pts

Source Code Review

Code is commented at the top, for each function, and as needed	_____ 20 pts
Code uses reasonable and consistent variable naming conventions	_____ 30 pts
The specified functions are present and are constructed in a logical manner	_____ 60 pts
The specified objects are present and are constructed in a logical manner	_____ 60 pts
File error handling is performed properly	_____ 40 pts
Depth-First Search and Pathfinding is implemented properly	_____ 60 pts
Main function performs only reporting functionality	_____ 40 pts

Total Points: 500 pts

Source Code:

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>

using namespace std;

struct coords
{
    int x;
    int y;
};

class level
{
public:
    vector<vector<char>> _map;

    level()
    {

    }

    void render(coords start)
    {
        for (int i = 0; i < _map.size(); i++)
        {
            for (int j = 0; j < _map[i].size(); j++)
            {
                if (i == start.x && j == start.y)
                {
                    cout << '@';
                }
                else
                {
                    cout << _map[i][j];
                }
            }
            cout << endl;
        }
    }

    static bool traverse(int i, int j, level lev, vector<coords>& path)
    {
        bool found = false;

        if (lev._map[i][j] == 'o')
        {
            cout << "End found at " << j << ", " << i << endl;
            found = true;
        }
        else if (lev._map[i][j] == '.')
        {
            lev._map[i][j] = 'x';

            if (i < lev._map.size() - 1)
            {
```

```
        found |= (traverse(i + 1, j, lev, path));
    }
    if (j < lev._map[i].size() - 1 && !found)
    {
        found |= (traverse(i, j + 1, lev, path));
    }
    if (i > 0 && !found)
    {
        found |= (traverse(i - 1, j, lev, path));
    }
    if (j > 0 && !found)
    {
        found |= (traverse(i, j - 1, lev, path));
    }
}

if (found)
{
    coords c = { c.x = i, c.y = j };
    path.push_back(c);
}

return found;
}

bool load(string path, coords& start)
{
    string line = "";
    ifstream file(path);

    if (file.is_open())
    {
        int j = 0;
        while (file >> line)
        {
            _map.push_back(vector<char>());
            for (int i = 0; i < line.length(); i++)
            {
                if (line[i] == '@')
                {
                    start.x = j;
                    start.y = i;
                    line[i] = '.';
                }

                _map[j].push_back(line[i]);
            }
            j++;
        }
        return true;
    }
    else
        return false;
}

};

int main()
{
    coords start;
```

```
vector<coords> path = vector<coords>();  
level lev;  
  
lev.load("map.txt", start);  
  
lev.render(start);  
  
cout << endl;  
  
level::traverse(start.x, start.y, lev, path);  
  
cout << endl;  
  
for (int i = path.size()-1; i >= 0; i--)  
{  
    cout << "Step " << path.size()-i << ": " << path[i].y << ", " << path[i].x  
<< endl;  
}  
  
}
```